



Aplicaciones web 2

Analista de Sistemas y Desarrollo de Software

Proyecto Web - "Pizzería Lobato"

Informe de desarrollo

Integrantes

Mariano Alberto López
Facundo Elgueta Millán

INDICE

1. Información recolectada (TP1)	2
2. Equipo y roles (TP1).....	3
3. ¿Qué se va a desarrollar? (TP1).....	3
4. Cronograma y organización de tareas (TP1)	4
5. Seguimiento y control de versiones (TP1)	5
6. Back-end para servir el sitio web (TP2).....	6
7. Creación del CRUD para administrar Front-End (TP3)	9

1. Información recolectada (TP1)

Ficha

- **Nombre:** Pizzería Lobato
- **Tipo de negocio:** Gastronómico – Elaboración y venta de pizzas y empanadas
- **Ubicación:** Buenos Aires, Argentina
- **Contacto:** Tel: +54 011 46418269
- **Persona designada para brindar información:** Sebastián Maciel – Correo: maciel_77@live.com.ar – Tel: +54 011 37931972

Sobre Pizzería Lobato

Pizzería Lobato es una reconocida marca de barrio que ya cuenta con clientes fieles desde sus inicios.

Nace con Felipe Lobato que era conocido en el barrio como el mejor pizzero de la zona de La Matanza. Luego de su muerte en 1973, sus tres hijos decidieron seguir con el legado de su papá, abriendo en ese mismo año "Pizzería Lobato", donde comenzó siendo una simple pizzería de barrio y terminó siendo una de las más concurridas y recomendadas.

Pasados los años, como todo negocio, tuvo que renovarse por las inevitables innovaciones de la sociedad y cambios económicos del país. Por lo que dos de sus hijos se fueron a España donde continúan cocinando para la gente en un pueblo chico de Madrid. Quedando así solo Sebastián, que continúa con el legado de su padre para que el barrio y todo el partido de La Matanza no se queden sin las clásicas pizzas Lobato.

Lista de requerimientos funcionales

- Página de inicio con una breve presentación de la pizzería, mostrar algunos de los productos y promociones con imágenes destacadas.
- Mostrar a través de la página de inicio la dirección de la pizzería y la posibilidad de acceder directamente a las promociones y carta.
- Formulario de contacto con validación de campos.
- Diseño responsivo para desktop y móvil.
- Menú de navegación fijo y accesible.
- Sección de Carta interactiva con filtros.
- Animaciones suaves en scroll y hover para mejorar la experiencia.

2. Equipo y roles (TP1)

- ❖ López Mariano: Líder de proyecto.
- ❖ Elgueta Facundo: Desarrollador Fullstack.

** Al contar con solo dos integrantes los roles se pueden ir intercalando a medida que avance el proyecto.*

3. ¿Qué se va a desarrollar? (TP1)

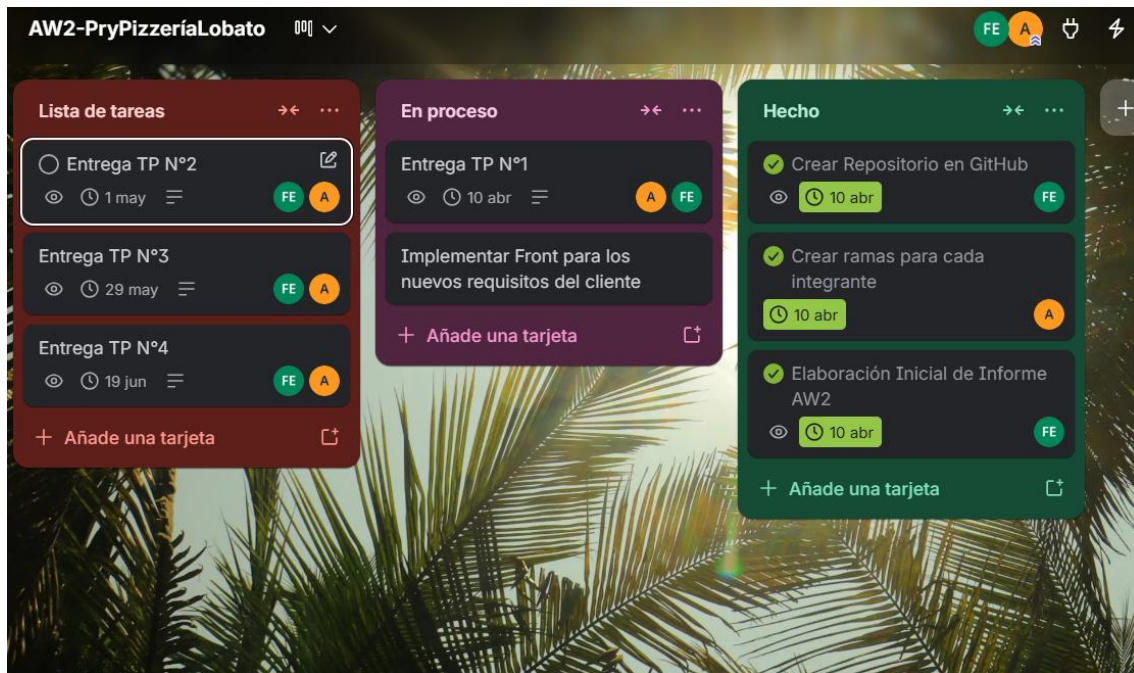
Actualmente se ha estado desarrollando una página “estática”, con esto queremos decir que el cliente solamente puede interactuar con la carta, pero no hay envío ni recibimiento de datos.

Se tiene pensado el desarrollo de una **gestión de productos**. La idea es que haya un administrador que pueda loguearse y poder: modificar los precios, imágenes, agregar o sacar productos.

4. Cronograma y organización de tareas (TP1)

Semana	Fecha	Desarrollo
1	23/03 al 27/03	Creación del repositorio GitHub.
2	30/03 al 03/04	Creación de las ramas con el nombre de cada integrante.
3	06/04 al 10/04	Elaboración de Informe y entrega del primer trabajo práctico.
4	13/04 al 17/04	Creación del servidor e instalación de nodeJS y expressJS.
5	20/04 al 24/04	Carga de datos en plataforma mockapi. Modularización del proyecto, desarrollo del informe.
6	27/04 al 01/05	Reunión presencial con el equipo, control de errores, rotación de roles y carga del html en servidor. Desarrollo del informe
7	04/05 al 08/05	Organización de proyectos utilizando patrón MVC.
8	11/05 al 15/05	Creación de APIs.
9	18/05 al 22/05	Creación interfaz front-end para administración.
10	25/05 al 29/05	Utilización de una base de datos. Reunión presencial del equipo para control de errores y rotación de roles. Desarrollo del informe.
11	01/06 al 05/06	Crear persistencia de datos en BBDD. Utilización del hashing.
12	08/06 al 12/06	Implementación de autenticación y control de acceso. Habilitar CORS en caso de ser necesario (Justificar).
13	15/06 al 19/06	Uso de variables de entorno, desarrollo del informe. Reunion presencial con el equipo y control de errores.

Se ha decidido como **herramienta** de seguimiento y organización de tareas utilizar **Trello**, debido a que ya existe cierta familiaridad en el uso de la misma.



Link: <https://trello.com/b/6jXaEqUX/aw2-prypizzerialobato>

5. Seguimiento y control de versiones (TP1)

Por último, se ha tomado la decisión de utilizar como gestor de versiones “GITHUB”.

Link: <https://github.com/Mariano-Lopez/TP-Lobato-AW2-Elgueta-Lopez>

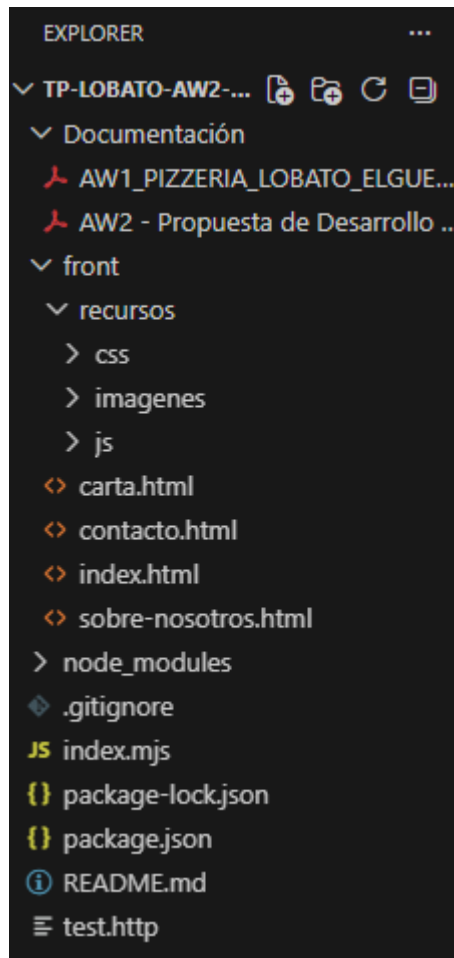
6. Back-end para servir el sitio web (TP2)

En esta etapa se desarrolló un servidor local, utilizando las tecnologías NodeJS y Express. Para lo cual se debió estructurar nuevamente el proyecto. Se creó una nueva carpeta “front” en la raíz, donde se guarda toda la interfaz del sitio, incluyendo HTML, CSS, JS e imágenes.

Nueva estructura del proyecto

A continuación, se detallará cada archivo y carpeta:

- **/index.mjs:** Archivo principal donde se ejecutará el servidor local y donde se importan los módulos *Express* y *path*.
- **/test.http:** Archivo donde se controla que los endpoints se ejecuten correctamente.
- **/README.md:** Archivo para guardar anotaciones.
- **/package.json:** Archivo creado por el módulo *Express*, donde se debe configurar el “type” en modules y en script configurar los identificadores al momento de ejecutar npm.
- **/package-lock.json:** Archivo creado por modulo *Express*.
- **/.gitignore:** Archivo git donde se especifica que carpeta debe ignorar al momento de subir los cambios en el repositorio, lo cual, en este caso, es la carpeta “node_modules”
- **front/recursos:** Contiene a su vez 3 carpetas con toda la configuración de CSS, JS y la última siendo la que almacena las imágenes del sitio completo.
- **front/index.html:** Archivo HTML de inicio del sitio.
- **front/carta.html:** Archivo HTML de la carta del sitio.
- **front/sobre-nosotros.html:** Archivo HTML con historia del negocio del sitio.
- **front/contacto.html:** Archivo HTML de contacto del sitio.



(nueva estructura del proyecto)

Tecnologías

Para el desarrollo de esta etapa se ha decidido utilizar las siguientes tecnologías:

- **NodeJS**: Entorno de ejecución de JavaScript de código abierto y multiplataforma que permite ejecutar código JS fuera del navegador web, principalmente en servidores.
- **Express**: Es el framework web estándar para Node.js, diseñado para crear aplicaciones web y APIs de forma rápida y flexible.
- **Path**: Es una utilidad integrada diseñada para gestionar y transformar rutas de archivos y directorios de forma segura entre diferentes sistemas operativos.
- **NPM**: Es el gestor de paquetes predeterminado para Node.js.

Datos de API externa (links mockApi)

- <https://69e68ec268208c1debe7aecb.mockapi.io/pizzeria-lobato/productos>
- <https://69e68ec268208c1debe7aecb.mockapi.io/pizzeria-lobato/promociones>

- Estructura de los datos

- **id**: Identificador único.
- **nombre**: Nombre del producto.
- **precio**: Monto del producto.
- **imagen**: Se divide nuevamente en 2 partes:
 - **src**: Ruta de archivo.
 - **alt**: En caso de no cargar imagen, texto descriptivo.
- **categoria**: Categoría a la que pertenece.

Desarrollo completo

El servidor como se mencionó anteriormente fue creado con modulo Express.

Se creó un puerto predeterminado para consultar, que en este caso es el 3000.

Se utiliza el método de express “.use” donde la búsqueda de la interfaz la realiza dentro de la carpeta *front*. Para acceder a las otras páginas del sitio se debe especificar la ruta exacta, por ejemplo: “http://localhost:3000/carta”.

Se debe declarar que el servidor escuche al puerto creado anteriormente mediante parámetro, utilizando el método “.listen”.

Cuando el cliente realiza la petición, cada modulo realiza el fetch correspondiente para devolver la información.

7. Creación del CRUD para administrar Front-End (TP3)

Para avanzar con el proyecto, se creó una base de datos denominada "carta", la cual contiene todos los productos del negocio.

```
DROP TABLE IF EXISTS carta;
>Run | ☐Select
CREATE TABLE carta (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio NUMERIC(10, 2) NOT NULL,
    categoria VARCHAR(50) NOT NULL,
    imagen VARCHAR(255)
);
```

Se decidió elegir como tipo de dato "VARCHAR" al campo de imagen, para que simplemente apunte a la ruta donde se aloja la imagen de determinado producto. Si el tipo de dato elegido fuera para almacenar de forma directa una imagen, a la larga afectaría negativamente al rendimiento de la base de datos, así como aumentar significativamente su tamaño.

Otro punto a detallar es la creación de la carpeta "/archivos", el propósito de esta carpeta es que sirva para almacenar nuevas imágenes de productos. Los productos ya existentes tienen su propia imagen por defecto en "/front-web/recursos/imagenes/", la idea es mantener una distinción entre las imágenes de productos ya existentes y las de nuevos productos.

El objetivo principal de esta parte del proyecto sería poder realizar un CRUD en los productos de la pizzería. Además, crear una nueva interfaz para que dicho CRUD pueda ser gestionado mediante un administrador, haciendo los cambios de modularización correspondiente e incorporar la arquitectura MVC.

Tecnologías

Junto con las tecnologías ya mencionadas anteriormente, se incorporaron nuevas, estas son:

- **Docker**: Plataforma de software que permite empaquetar aplicaciones y todas sus dependencias en contenedores.
- **PostgreSQL**: Es un sistema de gestión de bases de datos relacional (SGBDR) de código abierto.
- **Pg**: Librería oficial de Node.js para conectar y gestionar bases de datos PostgreSQL.
- **Pnpm**: Gestor de dependencias rápido y eficiente para proyectos JavaScript y Node.js.
- **Variables de entorno(.env)**: Es un archivo de texto simple que almacena configuraciones y datos confidenciales de una aplicación.
- **Multer**: Es un popular middleware de Node.js diseñado para manejar la carga de archivos en aplicaciones web.

Desarrollo completo

Se ha incorporado una nueva carpeta “__docker-pg” la cual contiene un instructivo de cómo proceder a la manipulación de la imagen. La misma también contiene a la base de datos “carta”, mencionada anteriormente.

Además, se creó otra nueva carpeta “bbdd” que contiene un archivo mjs denominado “conexión-bd.mjs”, dentro de este encontraremos los valores necesarios para poder realizar la conexión a la base de datos utilizando el módulo pg. La estructura se compone de:

- **Host**
- **User**
- **Password**
- **Database**
- **Port**

Estas contienen el valor que se declaran en el archivo de variables de entorno (.env).

Siguiendo con la estructura del proyecto, dentro de la carpeta “front-crud” se encuentran los archivos del front, donde el administrador podrá realizar el CRUD mencionado anteriormente. Se ha decidido utilizar IA (UNICAMENTE) para la confección del HTML, CSS y JS de este front.

ADMINISTRACIÓN - CRUD DE PRODUCTOS

Tipo de operación: **INSERTAR** MODIFICAR CONSULTAR ELIMINAR

● INGRESAR NUEVO PRODUCTO

Nombre Producto * Ingresar nombre de producto...

Precio Producto * Ingresar precio de producto...

Categoría * Seleccionar categoría...

Imagen del producto

Seleccionar imagen
JPG, PNG, WEBP - max 5 MB

* Campos requeridos

CONFIRMAR INSERCIÓN

En "/modulos/productos" se desarrolló el CRUD, se aplicó la arquitectura MVC y se crearon los archivos de controlador, modelo y rutas:

- **controlador.mjs**: Actúa como intermediario entre las rutas y el modelo. La mayor parte de la lógica está acá, recibe parámetros de URL (req.params) y el cuerpo de la petición (req.body), valida los datos que sean necesarios, procesa la respuesta del modelo y envía la respuesta HTTP al cliente.
- **modelo.mjs**: Interactúa directamente con la base de datos. Construye y ejecuta consultas SQL "SELECT, INSERT, UPDATE, DELETE" y retorna los resultados obtenidos al controlador.

- **rutas.mjs**: Aquí se definen los endpoints y se configuran los verbos (GET, POST, PUT, DELETE). Se asocian con la función del controlador que corresponda.

En resumen, el flujo es:

Cliente hace petición → Rutas dirige la petición a controlador → Controlador extrae los datos necesarios, valida y llama a modelo → Modelo ejecuta la consulta SQL y devuelve resultado → Controlador arma la respuesta JSON y esta misma le llega al cliente.

Para el último archivo (rutas.mjs), es necesario hacer unas aclaraciones para entender mejor su estructura y su funcionalidad:

```
// Obtener toda la carta (todos los productos)
rutasProductos.get('/api/v1/productos', controlador.obtenerCarta)

// Obtener todos los productos de una categoría
rutasProductos.get('/api/v1/productos/categoria/:categoria', controlador.obtenerTodos)

// Obtener un producto por id
rutasProductos.get('/api/v1/productos/:id', controlador.obtenerUno)

// Crear un producto
rutasProductos.post('/api/v1/productos', controlador.crearUno)

// Borrar un producto por id
rutasProductos.delete('/api/v1/productos/:id', controlador.borrarUno)

// Actualizar un producto por id
rutasProductos.put('/api/v1/productos/:id', controlador.actualizarUno)

export default rutasProductos
```

- **.get('/api/v1/productos')** = Obtiene toda la carta (todos los productos). Se ejecuta consulta SQL de SELECT con UNION, para unificar todas las tablas.
- **.get('/api/v1/productos/categoria/:categoria')** = Obtiene todos los productos de una categoría en específico.
- **.get('/api/v1/productos/:id')** = Obtiene solo un producto (mediante id).
- **.post('/api/v1/productos')** = Crea un producto nuevo.

- **.delete('/api/v1/productos/:id')** = Borra un producto existente (mediante id).
- **.put('/api/v1/productos/:id')** = Actualiza un producto existente (mediante id).

Aclaración: En la interfaz, el usuario no deberá de ingresar el id manualmente en ninguna operación a excepción del “Eliminar”. Si el usuario no sabe el id del producto que debe eliminar, entonces debe de realizar la operación de consulta.

En el nivel raíz del proyecto, se crearon los archivos:

- **.env**: Contiene las variables de entorno.
- **iniciar.env.mjs**: Ejecuta la configuración de dotenv (librería de desarrollo que permite cargar variables de entorno desde un archivo de texto).

Se ha modificado el archivo “index.mjs”, sumándole nuevas funcionalidades:

- **express.static()**: le dice a Express que sirva archivos (HTML, CSS, JS, imágenes) sin procesamiento adicional.
- **app.use(express.static(path.resolve('./front-web')))**: La carpeta /front-web se sirve en la raíz (/) por defecto. Se muestra el sitio web al entrar en "http://localhost:3000".
- **app.use('/admin', express.static(path.resolve('./front-crud')))**: La carpeta /front-crud se sirve mediante la ruta "/admin", la interfaz para administración se hace visible al entrar en <http://localhost:3000/admin>.

8. Implementación de autenticación, accesos y seguridad (TP4)

En esta instancia final, se ha decidido incorporar e implementar herramientas de seguridad para el manejo de datos de usuario, en este caso, en el inicio de sesión y CRUD de productos.

Se ha modificado la base de datos “carta”, y se la ha renombrado a “admin”, ya que la información que se contempla no es solamente de productos sino también ahora de usuarios.

```
▷Run
DROP TABLE IF EXISTS carta;
▷Run | □Select
CREATE TABLE carta (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio NUMERIC(10, 2) NOT NULL,
    categoria VARCHAR(50) NOT NULL,
    imagen VARCHAR(255)
);

▷Run
DROP TABLE IF EXISTS usuarios;
▷Run | □Select
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    username VARCHAR(150) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    session_id VARCHAR(255) UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

La base datos ahora cuenta con 2 tablas. La primera es “carta”, donde se almacena información relacionada con los productos en sí, la segunda tabla es “usuarios”, donde se guardan las credenciales de los mismos.

Se dejó cargado por defecto solamente 2 usuarios, que son los administradores que pueden realizar el CRUD completo sobre los productos de la pizzería.

```
▷Run | □Select
INSERT INTO usuarios (username, password_hash) VALUES
    ('Mariano-Lopez', '$2b$10$UYVXMC5dE0ZOP1Fs2RPTQuvNqSAbW9vbhixJ8eL7MepEDxnCmuku'), -- contraseña: admin1
    ('Facundo-Elgueta', '$2b$10$MmR.kNTWw4aqfth2QvviyeJ7CXH7vAxLFP4xCKB1ZZQqW9Xuj6Lji') -- contraseña: admin2
```

Se ha tomado la decisión de que solo los administradores puedan tener acceso y gestionar el CRUD. Es por esta razón que no se va a permitir el registro de usuarios.

Tecnologías

Se han incorporado 3 nuevos paquetes que establecen seguridad a la autenticación de los usuarios.

- **Bcryptjs**: Librería de JavaScript utilizada para el hashing (encriptación) seguro de contraseñas.
- **Jsonwebtoken**: Es un estándar abierto que se utiliza para transmitir información de forma segura y compacta entre dos partes en formato JSON.
- **Cookie-parser**: Es un middleware para NodeJs diseñado para analizar (parsear) las cookies enviadas en las peticiones HTTP entrantes.

Desarrollo completo

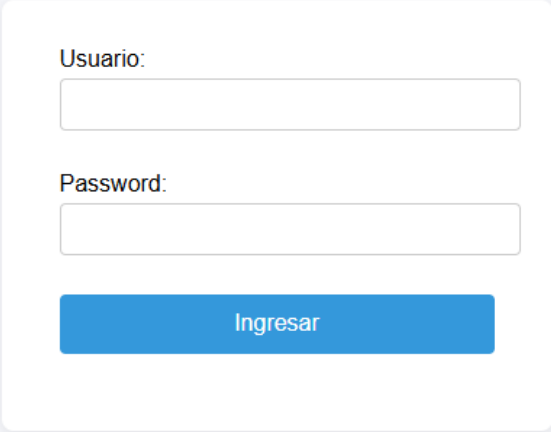
El proyecto cuenta ahora con la carpeta “/fronts”, con el objetivo de organizar un poco más la estructura del proyecto. Esta carpeta contiene:

- **/front-web**: Contiene los archivos e interfaz pública del negocio.
- **/front-login**: Almacena los archivos e interfaz para la autenticación del administrador en la gestión de productos (CRUD).
- **/front-crud**: Carpeta con archivos e interfaz, que luego de haberse autenticado le permitirá al administrador gestionar los productos del negocio.

En esta última etapa del proyecto se ha trabajado con el manejo de información sensible. Como se ha mencionado anteriormente la base de datos cuenta con una tabla “usuarios”, la cual dentro de la misma se almacena, entre otros datos, “username” y “password_hash”.

Como primer paso de la autenticación, se chequea el **usuario** que envía el administrador, consultando su existencia en la base de datos. En caso de que no exista o la información ingresada sea errónea, se le avisa al mismo mediante un mensaje en la interfaz.

La **contraseña** de los administradores se las ha encriptado y guardado en la base de datos para mayor seguridad (Este proceso se logra gracias al paquete bcryptjs). Como segundo paso de la autenticación, este proceso se debe realizar de forma inversa, se obtienen la contraseña encriptada de la base de datos para poder hacer la comparación con la contraseña que envía el administrador.



Usuario:

Password:

Ingresar

Si todo sale bien y los datos fueron correctos, el administrador es redirigido automáticamente a la ruta “/admin”, donde se encontrará con la interfaz del CRUD de productos. Una vez logueado, se creará una cookie y un token que permitirán la navegación en la pagina durante un periodo de tiempo, en este caso hemos establecido un tiempo de 5 minutos.

Si el usuario terminó con su cometido antes del tiempo establecido, si lo desea, podrá hacer click en el botón “Cerrar Sesión”, lo cual dará de baja la cookie y token generados. En el caso de querer volver a la pestaña anterior (el CRUD), o simplemente quiere realizar otra operación cuando el tiempo ya ha finalizado, será redireccionado de nuevo al login y deberá volver a ingresar sus credenciales.